

```

import numpy as np
import matplotlib.pyplot as plt
import json
import sys
import matplotlib.pyplot
import scipy.linalg

import plotly
import plotly.graph_objs as go

```

Introduction to numerical methods

János Török

Class 3: Linear algebra 2.

System of linear equations

The general form is:

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + \dots + a_{1n}x_n = b_1 \\ a_{21}x_1 + a_{22}x_2 + \dots + a_{2n}x_n = b_2 \\ \vdots \\ a_{m1}x_1 + a_{m2}x_2 + \dots + a_{mn}x_n = b_m \end{cases}$$

where $x_1, x_2, \dots, x_n$ are the unknowns, $a_{11}, a_{12}, \dots, a_{mn}$ are the coefficients of the system, and $b_1, b_2, \dots, b_m$ are the constant terms.

In matrix form

$$Ax = b,$$

where

$$A = \begin{pmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{pmatrix}, x = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix}, b = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{pmatrix}.$$

Gauss elimination

First we create a matrix $(A \vee b)$. Then we try to make the value below the diagonal zero, and the diagonal one. Allowed steps:

- Swapping two rows,
- Multiplying a row by a nonzero number,
- Adding a multiple of one row to another row.

Example:

$$\begin{pmatrix} 1 & 3 & 1 & 9 \\ 1 & 1 & -1 & 1 \\ 3 & 11 & 5 & 35 \end{pmatrix} \rightarrow \begin{pmatrix} 1 & 3 & 1 & 9 \\ 0 & -2 & -2 & -8 \\ 0 & 2 & 2 & 8 \end{pmatrix} \rightarrow \begin{pmatrix} 1 & 3 & 1 & 9 \\ 0 & -2 & -2 & -8 \\ 0 & 0 & 0 & 0 \end{pmatrix} \rightarrow \begin{pmatrix} 1 & 0 & -2 & -3 \\ 0 & 1 & 1 & 4 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

The steps:

1. The first row times -1 is added to the second row, and the -3 times the first row is added to the third row
2. The second row is added to the third row resulting in a row with only zeros, which is a singular case.
3. The second row is multiplied by -2 and it is added to the first row multiplied by -3 .

Implementation

```
A = np.array([[1,3,1],[1,1,-1],[3,11,5]],dtype=float)
b = np.array([9,1,35],dtype=float)
```

```
B = np.column_stack((A,b))
print(B)
```

```
[[ 1.  3.  1.  9.]
 [ 1.  1. -1.  1.]
 [ 3. 11.  5. 35.]]
```

```
def GE_swap(B,i,j):
    tmp = np.copy(B[i])
    B[i] = np.copy(B[j])
    B[j] = tmp
```

```
GE_swap(B,1,2)
print(B)
B = np.column_stack((A,b))
```

```
[[ 1.  3.  1.  9.]
 [ 3. 11.  5. 35.]
 [ 1.  1. -1.  1.]]
```

```
def GE_multiply(B,i,alpha):
    B[i] = B[i] * alpha
```

```
GE_multiply(B,0,0.5)
print(B)
B = np.column_stack((A,b))
```

```

[[ 0.5  1.5  0.5  4.5]
 [ 1.   1.  -1.   1. ]
 [ 3.  11.   5.  35. ]]

def GE_add_multiply(B,i,j,alpha):
    B[i] += B[j] * alpha

GE_add_multiply(B,1,0,-1)
print(B)

[[ 1.   3.   1.   9.]
 [ 0.  -2.  -2.  -8.]
 [ 3.  11.   5.  35.]]

```

Gauss elimination algorithm

1. Iterate for each column, except for the last (i)
2. In column i find an element in rows at or below i which are nonzero, and swap the rows if necessary
3. Clear all elements in column i below i In the end we should end up with a bottom triangle which is zero

```

#A safe point where we recreate the matrix and start our development
B = np.column_stack((A,b))
#GE_add_multiply(B,0,1,1)
#GE_swap(B,0,2)
print(B)
N = len(B)

[[ 1.   3.   1.   9.]
 [ 1.   1.  -1.   1.]
 [ 3.  11.   5.  35.]]

#column i from row i
i = 0
B[i:,i]

array([1., 1., 3.])

np.where(B[i:,i] > 0)[0]

array([0, 1, 2])

#Check the topmost row where there are nonzero elements, and give an error message if there aren't any
if len(np.where(B[i:,i] > 0)[0]) == 0:
    print("Singular matrix")
else:
    j = np.where(B[i:,i] > 0)[0].min()
    print(j)

0

```

```

#if the nonzero is in an other row, swap the two
if i != j:
    GE_swap(B,i,j)
print(B)

[[ 1.  3.  1.  9.]
 [ 1.  1. -1.  1.]
 [ 3. 11.  5. 35.]]

####
#Now the Gauss-elimination
####
#make the killer element 1
GE_multiply(B,i,1.0/B[i,i])
for j in range(i+1,N):
    GE_add_multiply(B,j,i,-B[j,i])
print(B)

[[ 1.  3.  1.  9.]
 [ 0. -2. -2. -8.]
 [ 0.  2.  2.  8.]]

#Solution
# i in reverse order
#left hand side

```

Solution

$$B = \begin{pmatrix} 1 & 3 & 1 & 9 \\ 0 & 1 & 1 & 4 \\ 0 & 0 & 1 & 1 \end{pmatrix}$$

Expanded form:

$$\begin{aligned} x_3 &= 1 \\ x_2 &= 4 - 1 \cdot x_3 \\ x_1 &= 9 - 3 \cdot x_2 - 1 \cdot x_3 \end{aligned}$$

Solution is

$$x_i = b_i - \sum_{j=i+1}^N b_j x_j$$

If the vector of solution at the beginning is zero $x = (x_1, x_2, x_3) = (0, 0, 0)$, then we can rewrite it as:

$$x_i = b_i - b x,$$

provided we go with i in the reverse order.

```

def my_GE(A,b):
    B = np.array(np.column_stack((A,b)),dtype=float)
    N = len(B)
    for i in range(N):
        if B[i,i] == 0.0:
            if len(np.where(B[i:,i] > 0)[0]) == 0:
                raise Exception("Singular matrix")
            else:
                j = np.where(B[i:,i] > 0)[0].min()
                GE_swap(B,i,j)
                GE_multiply(B,i,1.0/B[i,i])
            for j in range(i+1,N):
                GE_add_multiply(B,j,i,-B[j,i])
    sol = np.zeros(N,dtype=float)
    for i in reversed(range(N)):
        sol[i] = B[i,-1] - (np.dot(B[i,:N],sol)).sum()
    return sol

```

```

A = np.array([[1,3,1],[1,1,-1],[3,11,5]],dtype=float)
b = np.array([9,1,35],dtype=float)
my_GE(A,b)

```

```

-----
Exception                                Traceback (most recent call
last)

```

```

<ipython-input-20-4ee1362d1291> in <module>
      1 A = np.array([[1,3,1],[1,1,-1],[3,11,5]],dtype=float)
      2 b = np.array([9,1,35],dtype=float)
----> 3 my_GE(A,b)

```

```

<ipython-input-19-17d974c6d6ef> in my_GE(A, b)
      5         if B[i,i] == 0.0:
      6             if len(np.where(B[i:,i] > 0)[0]) == 0:
----> 7                 raise Exception("Singular matrix")
      8             else:
      9                 j = np.where(B[i:,i] > 0)[0].min()

```

```

Exception: Singular matrix

```

```

A = np.array([[1,3,1],[1,1,-1],[3,10,5]],dtype=float)
b = np.array([9,1,32],dtype=float)
x = my_GE(A,b)
print(x)

```

```

[-1.  3.  1.]

```

LU decomposition

The matrix A is decomposed into an L lower triangle and U upper triangle matrix such that $A=LU$. Example:

$$\begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{pmatrix} = \begin{pmatrix} l_{11} & 0 & 0 \\ l_{21} & l_{22} & 0 \\ l_{31} & l_{32} & l_{33} \end{pmatrix} \begin{pmatrix} u_{11} & u_{12} & u_{13} \\ 0 & u_{22} & u_{23} \\ 0 & 0 & u_{33} \end{pmatrix}$$

In particular a common choice is that:

$$L = \begin{pmatrix} 1 & 0 & 0 \\ l_{21} & 1 & 0 \\ l_{31} & l_{32} & 1 \end{pmatrix},$$

in which case the decomposition is unique.

Simple solution:

Use Gauss elimination to get L , in a form $Lx=x$:

$$\begin{pmatrix} 1 & 0 & \dots & 0 \\ l_{21} & 1 & \dots & 0 \\ \vdots & \vdots & \ddots & 0 \\ l_{n1} & l_{n2} & \dots & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{pmatrix}$$

Such a matrix can be obtained by writing a Gauss elimination algorithm upside down. The U matrix can be obtained by performing the matrix multiplication of LU and going through in a typewriter style:

The first line:

$$\begin{array}{l} a_{11} \quad \textcolor{red}{l}1 \cdot u_{11} \\ a_{12} \quad \textcolor{red}{l}1 \cdot u_{12} \\ \vdots \quad \textcolor{red}{l}\vdots \\ a_{1n} \quad \textcolor{red}{l}1 \cdot u_{1n} \end{array}$$

So the first line is the same as that of A . The second line:

$$\begin{array}{l} a_{21} \quad \textcolor{red}{l}l_{21} \cdot u_{11} \\ a_{22} \quad \textcolor{red}{l}l_{21} \cdot u_{12} + l_{22} \cdot u_{22} \\ \vdots \quad \textcolor{red}{l}\vdots \\ a_{2n} \quad \textcolor{red}{l}\sum_j l_{2j} \cdot u_{jn} \end{array}$$

We can see that we only use parts of the U matrix which is already known.

The algorithm is not too difficult we could use our Gauss elimination but we will not implement it. The reason is the possibility of singularity during the process requires pivoting elements (changing rows and columns). One has to keep track of that in three matrices, in L , in the actually created U and a pivot matrix P , which is defined as $PA = UL$. In general the procedure is done in a different order. First the pivot matrix is determined from the maximum values of the A matrix. If only partial pivoting is done, then the one after the other the row with the highest element in the diagonal is swapped to the actual position, in the case of full pivoting also columns are swapped. The pivoting procedure is coded in the pivot matrix.

```
A = np.array([[1,3,1],[1,1,-1],[3,10,5]],dtype=float)
print(A)
P,L,U = scipy.linalg.lu(A)

[[ 1.  3.  1.]
 [ 1.  1. -1.]
 [ 3. 10.  5.]]

P,L,U
(array([[0., 0., 1.],
        [0., 1., 0.],
        [1., 0., 0.]]),
 array([[1., 0., 0.],
        [0.33333333, 1., 0.],
        [0.33333333, 0.14285714, 1.]]),
 array([[ 3., 10.,  5.],
        [ 0., -2.33333333, -2.66666667],
        [ 0.,  0., -0.28571429]]))

np.linalg.inv(P).dot(np.dot(L,U))

array([[ 1.,  3.,  1.],
       [ 1.,  1., -1.],
       [ 3., 10.,  5.]])
```

Iterative methods: Gauss-Seidel

Sometimes iterative methods are desirable. Especially if there are limit issues, e.g. there are too small and too large variables, or if e.g. the matrix is a result of a dynamical process and changes slowly and so the solutions, than iterative methods can be much faster. An other important use case is the sparse matrices, where creating an algorithm which uses this nature to its advantage is easy with iterative methods but impossible with Gauss elimination.

Let us express the components of x from the set of linear equations in an iterative form, namely supposing we know all other components:

$$Ax=b$$

In components:

$$\begin{array}{rcl}
 a_{11}x_1 + a_{12}x_2 + \dots + a_{1n}x_n & \hat{=} & b_1 \\
 a_{21}x_1 + a_{22}x_2 + \dots + a_{2n}x_n & \hat{=} & b_2 \\
 \vdots & \hat{=} & \vdots \\
 a_{n1}x_1 + a_{n2}x_2 + \dots + a_{nn}x_n & \hat{=} & b_n
 \end{array}$$

The different components can be expressed using superscript for iteration count:

$$\begin{array}{rcl}
 x_1^{(i+1)} & \hat{=} & \frac{-1}{a_{11}} (a_{12}x_2^{(i)} + \dots + a_{1n}x_n^{(i)} - b_1) \\
 x_2^{(i+1)} & \hat{=} & \frac{-1}{a_{22}} (a_{21}x_1^{(i)} + a_{23}x_3^{(i)} + \dots + a_{2n}x_n^{(i)} - b_2) \\
 \vdots & \hat{=} & \vdots \\
 x_n^{(i+1)} & \hat{=} & \frac{-1}{a_{nn}} (a_{n1}x_1^{(i)} + a_{n2}x_2^{(i)} + \dots + a_{n,n-1}x_{n-1}^{(i)} - b_n)
 \end{array}$$

Please note the minus sign in the nominator of the first term on the right hand side of the equation.

This is the Jordan method, so what can we do better? Why not to use the already calculated values. It was shown that it converges faster.

$$\begin{array}{rcl}
 x_1^{(i+1)} & \hat{=} & \frac{-1}{a_{11}} (a_{12}x_2^{(i)} + \dots + a_{1n}x_n^{(i)} - b_1) \\
 x_2^{(i+1)} & \hat{=} & \frac{-1}{a_{22}} (a_{21}x_1^{(i+1)} + a_{23}x_3^{(i)} + \dots + a_{2n}x_n^{(i)} - b_2) \\
 \vdots & \hat{=} & \vdots \\
 x_n^{(i+1)} & \hat{=} & \frac{-1}{a_{nn}} (a_{n1}x_1^{(i+1)} + a_{n2}x_2^{(i+1)} + \dots + a_{n,n-1}x_{n-1}^{(i+1)} - b_n)
 \end{array}$$

In form of a sum:

$$x_j^{(i+1)} = \frac{-1}{a_{jj}} \left(\sum_{k=1}^{j-1} a_{jk} x_k^{(i+1)} + \sum_{k=j+1}^n a_{jk} x_k^{(i)} - b_j \right)$$

The last but most important thing to mention is that this method converges only if the matrix A is either diagonally dominant, positive definite, or symmetric. Let us first write the algorithm keeping in mind that all iterative methods need starting values.

```

#once again work step by step
A = np.array([[3,1,1],[3,10,5],[1,-1,1]],dtype=float)
b = np.array([1,32,-3],dtype=float)
N = len(b)
my_GE(A,b)

array([-1.,  3.,  1.])

```



```

#initial values
x0 = np.ones(N,dtype=float)
x = np.copy(x0)

#iteration, here it is very simple
for j in range(N):
    x[j] = -(np.sum(A[j] * x) - A[j,j] * x[j] - b[j]) / A[j,j]
    print(x)

[-0.33333333  1.          1.          ]
[-0.33333333  2.8         1.          ]
[-0.33333333  2.8         0.13333333]

# stopping conditions
prev_x = np.copy(x)
for j in range(N):
    x[j] = -(np.sum(A[j] * x) - A[j,j] * x[j] - b[j]) / A[j,j]
print(np.linalg.norm(x-prev_x))

1.0373232409296727

#Besides A and b we need three parameters, the stopping limit, maximum
iteration and the initial condition
def my_GS(A, b, lim = 1e-3, max_iter = 1000, x0 = []):
    #if init is not given we will start from the vector (1,1,...,1)
    N = len(b)
    if len(x0) != N:
        x0 = np.ones(N,dtype=float)
    x = np.copy(x0)
    for i in range(max_iter):
        prev_x = np.copy(x)
        for j in range(N):
            x[j] = -(np.sum(A[j] * x) - A[j,j] * x[j] - b[j]) / A[j,j]
            if np.linalg.norm(x-prev_x) < lim:
                break
        return x,i,np.linalg.norm(x-prev_x)

#once again work step by step
A = np.array([[3,1,1],[3,10,5],[1,-1,1]],dtype=float)
b = np.array([1,32,-3],dtype=float)

x, niter,lim = my_GS(A,b)
print(x)
print(niter,lim)

[-0.99967631  3.00021519  0.9998915 ]
9 0.0007641281018185905

%%timeit
A = np.array([[3,1,1],[3,10,5],[1,-1,1]],dtype=float)

```

```
b = np.array([1,32,-3],dtype=float)
my_GE(A,b)
```

36.6 μ s $\pm$ 350 ns per loop (mean $\pm$ std. dev. of 7 runs, 10000 loops each)

```
%%timeit
A = np.array([[3,1,1],[3,10,5],[1,-1,1]],dtype=float)
b = np.array([1,32,-3],dtype=float)
my_GS(A,b)
```

219 μ s $\pm$ 10.5 μ s per loop (mean $\pm$ std. dev. of 7 runs, 1000 loops each)

```
A = np.array([[1,3,1],[1,1,-1],[3,10,5]],dtype=float)
b = np.array([9,1,32],dtype=float)
x, niter,lim = my_GS(A,b)
print(x)
print(niter,lim)
```

```
[ 8.69234435e+191  1.87447450e+192 -4.27048967e+192]
999 inf
```

#symmetric matrix

```
A = np.array([[1,3,1],[1,4,2],[1,2,4]],dtype=float)
b = np.array([9,13,9],dtype=float)
x, niter,lim = my_GS(A,b)
print(x)
print(niter,lim)
```

```
[-0.99705189  2.99939451  0.99956572]
16 0.0003525996154096707
```